University of Bedfordshire

School of Computer Science and Technology

SplitMS – Expense Splitting Application

OOP Architecture and Implementation

Student Name: Aanand Thakkur
Student ID: 2542146
Module: CIS096-1
Submission Date: March 19, 2026

University of Bedfordshire

March 19, 2026

Abstract

This document discusses object-oriented architecture and the implementation of SplitMS, an expense-cutting device that was supposed to be used for expense-sharing with auditable shared-finance control. This document converts the original idea into a layered Java solution involving UI, controller, storage, repository, and database issues, as well as databases, which have a clear hierarchy of roles and dependencies. It evidences the fundamental OOP principles through service and repository contracts and polymorphism through interface-driven design. This document also addresses key design issues such as Singleton, Repository, Service Locator, and Result Object use, as well as practical trade-offs in maintainability and testability. It reveals the UI's key business flows, including user registration, expense estimation, safe password hashing, debt settlement coordination, and a shell-content navigator to show implementation depth. Finally, integration test coverage is documented, and implementation challenges, such as atomic multi-repository operations, and applied solutions are discussed to show the learning process and architectural correctness, robustness, and readiness for future extension.

Contents

1 Introduction **1**

2 Architecture Diagrams **1**

2.1 Layered Architecture Diagram 2

2.2 Package Structure Diagram 3

2.3 Sequence Diagram: createExpenseWithSplits Flow 4

3 Class Definitions **4**

3.1 Class Definition Template 5

3.2 Required Components Coverage Matrix 5

4 OOP Principles – Applied Evidence **6**

4.1 Encapsulation 6

4.2 Inheritance 6

4.3 Polymorphism 7

4.4 Abstraction 8

5 Design Patterns Used **8**

5.1 Singleton Pattern 8

5.2 Repository Pattern 9

5.3 Service Locator Pattern 9

5.4 Result Object (Railway-Oriented) Pattern 9

5.5 Shell + Content UI Composition Pattern 9

6 Implementation – Core Features **9**

6.1 Feature 1: User Registration with Automatic Personal Group Creation 9

6.2 Feature 2: createExpenseWithSplits and Atomic Split Validation 10

6.3 Feature 3: Password Hashing and Timing-Safe Verification 11

6.4 Feature 4: settleTransaction End-to-End Flow 11

6.5 Feature 5: ViewNavigator Shell + Content Swap Mechanism 11

7 Testing **12**

7.1 Test Class Coverage Matrix 12

7.2 Test Execution Evidence 13

7.3 Testing Limitation and Improvement Note 13

8 Challenges and Solutions **13**

8.1 Challenge 1: Atomic Split Creation Without Full Transaction Framework . . . 13

8.2 Challenge 2: JavaFX Shell + Content Lifecycle Coordination 14

A Appendix: Submission Repository **15**

List of Figures

1 Layered architecture of SplitMS (UI, Controller, Service, Repository, Database) 2

2 Package structure mapped to src layout (controllers, services, repositories, entities, models, lib, utils) 3

3 Sequence diagram for createExpenseWithSplits flow (Controller to Service to Repository to Database) 4

4 Full test suite execution output (green/passing) 13

List of Tables

1 Class documentation template 5
2 Mandatory Classes and Components 5
3 Integration test coverage overview 12

1 Introduction

SplitMS is an expense-cutting service that helps manage shared expenditures, assign split roles, and settle customer accounts in a simple and auditable manner. As a result of common collaborative scenarios such as flat sharing, travel groups, and event budgets where multiple users can contribute to or owe funds.

This Week 9 submission extends the Week 2 submission into an implementation-focused architecture document by showing how the domain model was translated into a layered Java application with clear service boundaries, repository abstractions, and database-backed persistence. The report focuses on object-oriented architecture, and evidence of fundamental OOP principles, design pattern application, and implementation of core transactional flows is shown through a sequence diagram.

2 Architecture Diagrams

Three diagrams are linked to the submission of the report in this section. Each diagram is shown with a caption and an explanation of the style chosen.

2.1 Layered Architecture Diagram

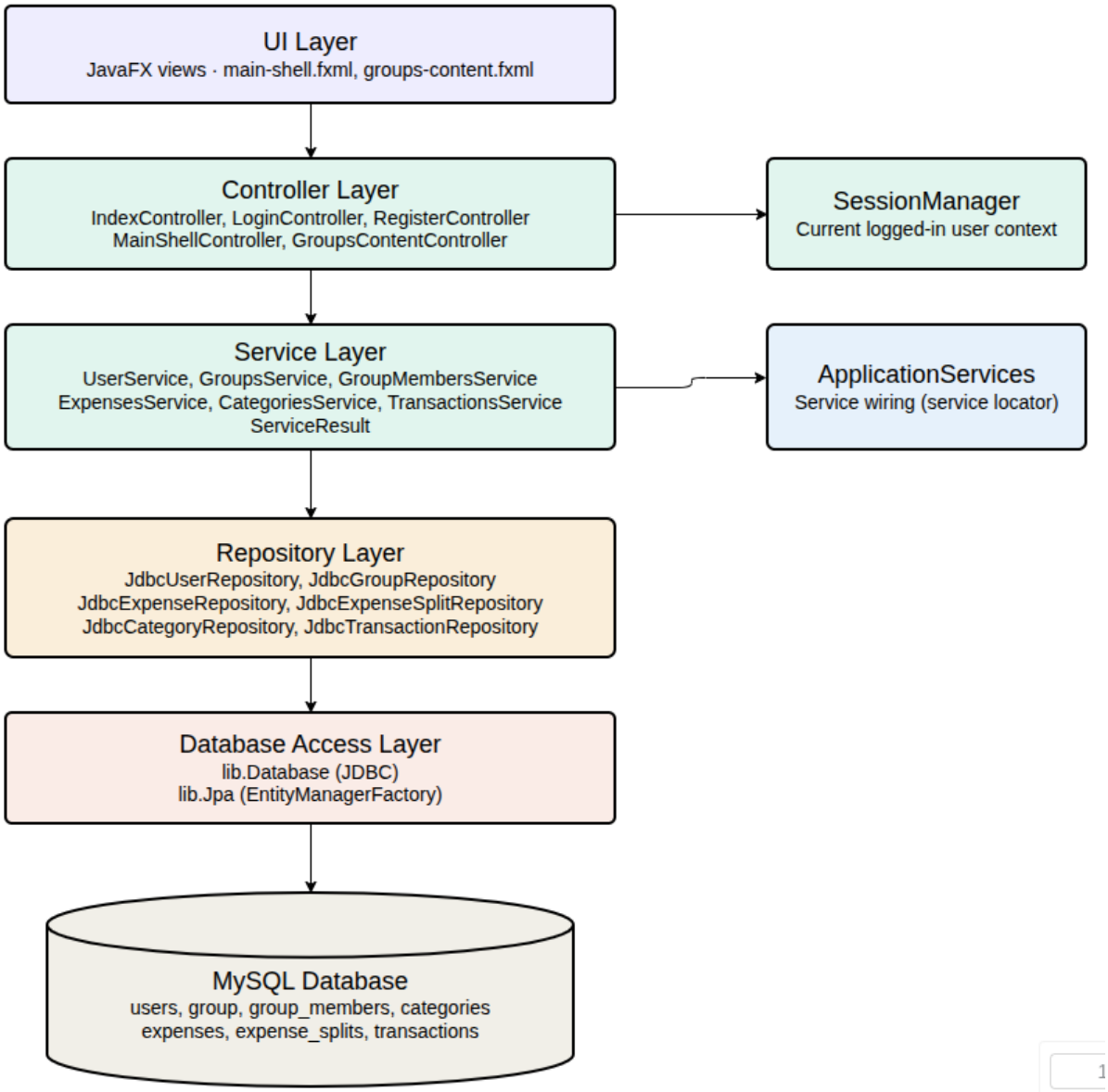


Figure 1: Layered architecture of SplitMS (UI, Controller, Service, Repository, Database)

The layered architecture separates responsibilities of the application across presentation, application logic, persistence abstraction, and storage layers. This separation reduces coupling and helps isolate business rules in the service layer, while repositories encapsulate data access information from controllers.

2.2 Package Structure Diagram

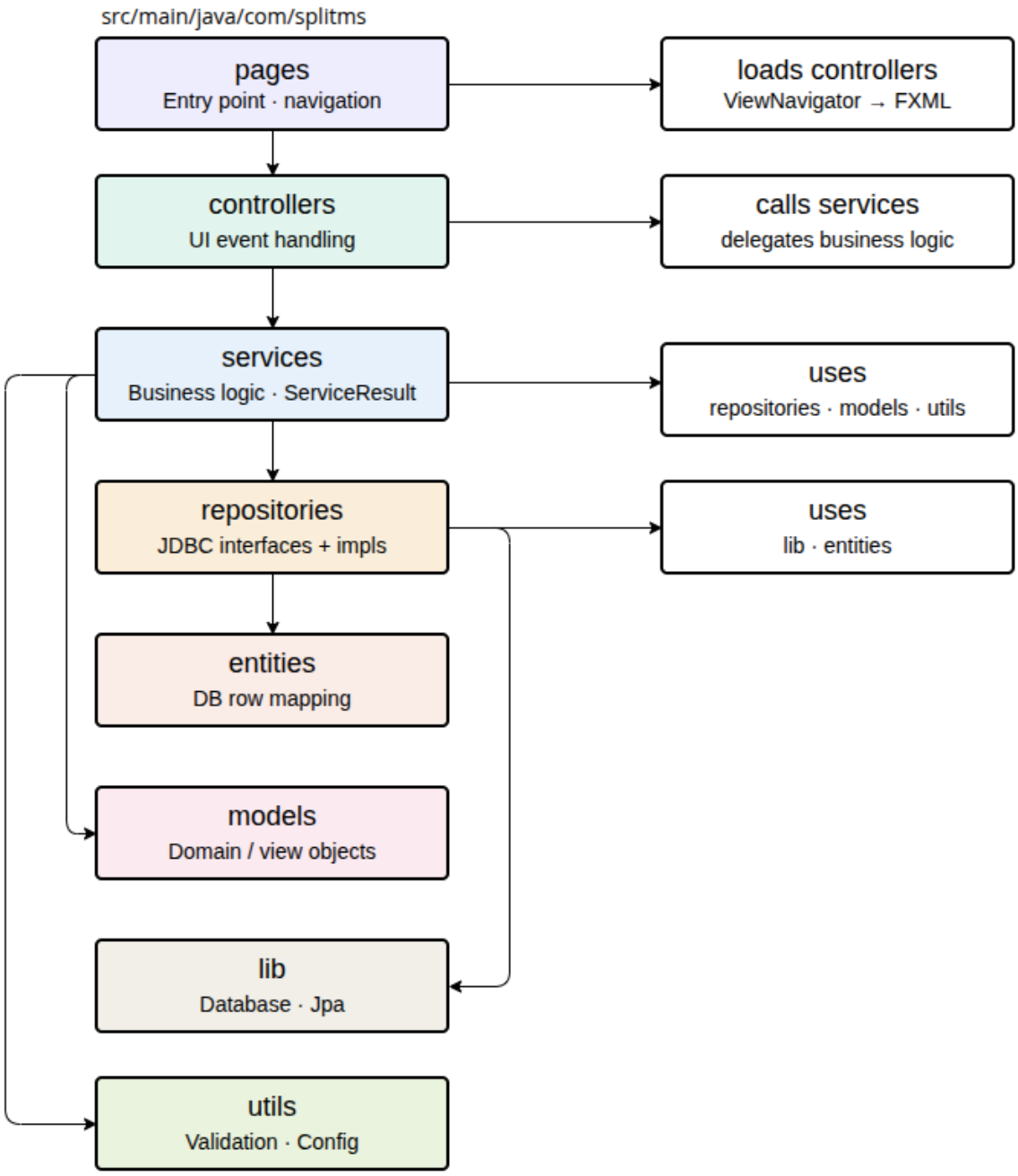


Figure 2: Package structure mapped to src layout (controllers, services, repositories, entities, models, lib, utils)

The package diagram links to the project's source layout and shows where each class and category belongs. This helps maintainability by making class ownership explicit and preventing company logic from leaking into controller or utility packages.

2.3 Sequence Diagram: createExpenseWithSplits Flow

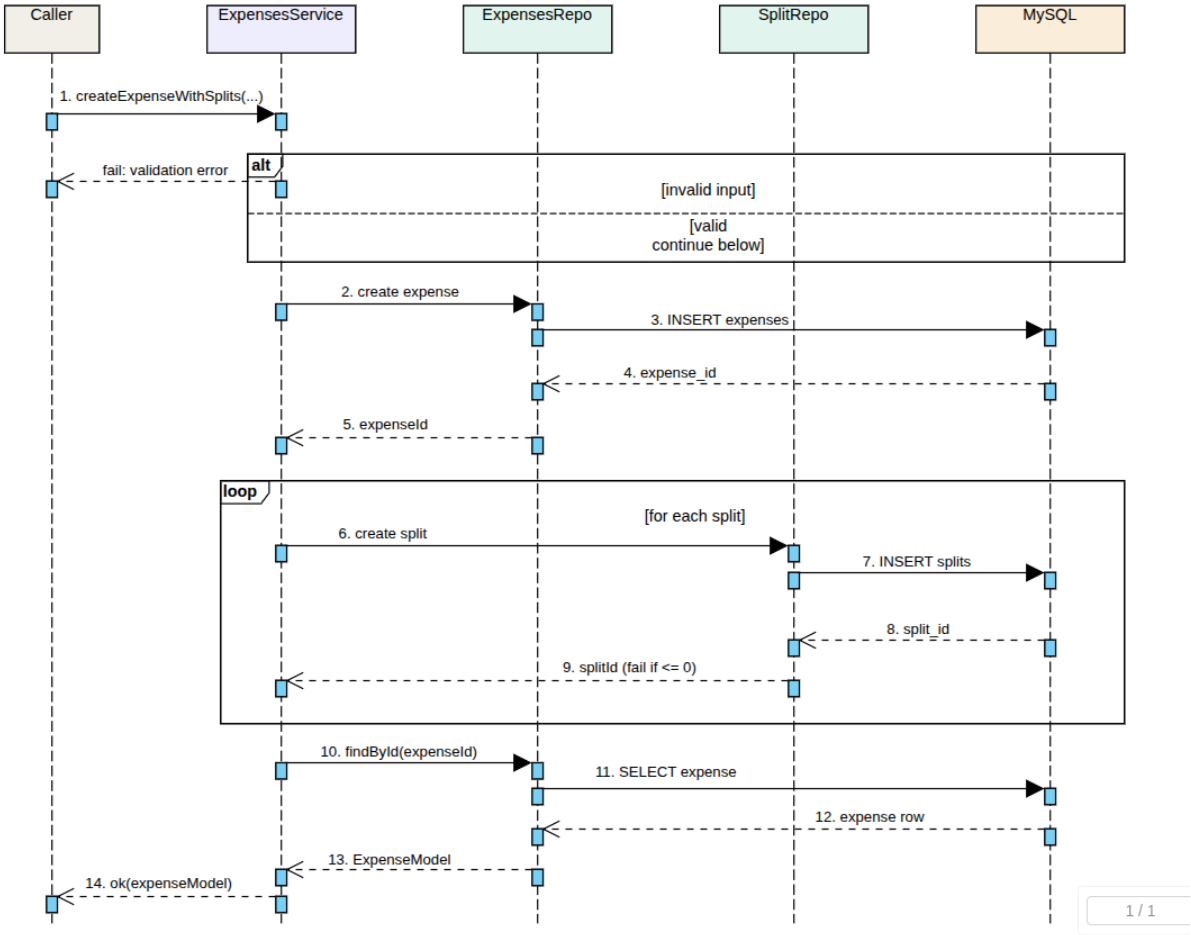


Figure 3: Sequence diagram for createExpenseWithSplits flow (Controller to Service to Repository to Database)

The sequence diagram shows validation, atomic coordination in several repositories, and result propagation back to the controller. This flow was selected because it is architecturally significant: it includes input verification, industry invariants, and multi-step persistence.

3 Class Definitions

This section contains key classes, interfaces, databases, and utility applications that do not have to produce complete source files. The table layout below follows the required class definition template.

3.1 Class Definition Template

Table 1: Class documentation template

Field	Description to Provide
Class name	Suitable class name.
Type	Class, interface, record, or enum.
Layer	UI, controller, service, repository, entity, model, lib, or utils.
Responsibility	One-sentence purpose.
Key attributes	Name, type, and visibility of important fields.
Key methods	Signature, return type, and purpose summary.
Relationships	Depends on, instantiates, extends, or implements.

3.2 Required Components Coverage Matrix

Table 2: Mandatory Classes and Components

Component	Type	Layer	Responsibility
SplitmsApplication	Class	Entry Point	App lifecycle and top-level wiring
ViewNavigator	Class	UI/Controller	Scene routing and shell switching
SessionManager	Singleton	Session	Logged-in user context
Database	Singleton	Persistence	JDBC connection lifecycle
ServiceResult<T>	Record	Service	Success/failure wrapper
ApplicationServices	Class	Service	Dependency wiring and service locator
UserService	Class	Service	Authentication and profile logic
GroupsService	Class	Service	Group creation and retrieval
GroupMembersService	Class	Service	Membership add/remove/list
ExpensesService	Class	Service	Splits, validation, settlements
TransactionsService	Class	Service	Transaction recording and debt flows
JdbcUserRepository	Class	Repository	User persistence (JDBC)
JdbcExpenseRepository	Class	Repository	Expense persistence (JDBC)
JdbcExpenseSplitRepository	Class	Repository	Split persistence (JDBC)
UserAccount	Record	Model	Immutable user projection
GroupModel	Record	Model	Immutable group projection
ExpenseModel	Record	Model	Immutable expense + amount data
ExpenseSplitModel	Record	Model	Immutable split with debtor/share
TransactionModel	Record	Model	Immutable settlement projection
GroupMemberModel	Record	Model	Immutable user-group mapping
SessionModel	Record	Model	Immutable session/login data

Table 2 (continued)

Component	Type	Layer	Responsibility
Pbkdf2PasswordHasher	Class	Lib	PBKDF2 hashing and verification
EnvConfig	Class	Utils	Environment config and defaults
Validation	Class	Utils	Input and domain rule checks
Normalize	Class	Utils	Canonical input normalization

4 OOP Principles – Applied Evidence

Each subsection below states the OOP principle and maps it to specific classes in SplitMS. Only essential snippets are included to keep the report concise.

4.1 Encapsulation

Encapsulation is made possible by immutable records such as `UserAccount` and `ExpenseModel`, where data is initialized once and exposed through accessors only. It is also visible in `SessionManager` and `Database`, where mutable internal state is hidden behind controlled methods.

Applied evidence (4.1): Record-based immutable models do not expose setters. Singleton classes keep private state through fields such as `private static final SessionManager INSTANCE` and controlled access via `getInstance()` and `currentUser()`. Interface-based hiding is also used: `PasswordHasher` exposes only `hash()` and `matches()`, while hashing algorithm details remain inside `Pbkdf2PasswordHasher`.

```java
public interface PasswordHasher {
    String hash(String plainTextPassword);
    boolean matches(String plainTextPassword, String storedHash);
}
```

Listing 1: Encapsulation – PasswordHasher interface

4.2 Inheritance

In controller and service contracts, inheritance (and interface inheritance) is present Examples include `NavigatorAware` implemented by controllers, `PasswordHasher` implemented by `Pbkdf2PasswordHasher`, and service interfaces built by concrete service classes.

Applied evidence (4.2): The interface methods `PasswordHasher#hash(String)` and `PasswordHasher#verify(String,String)` are implemented by `Pbkdf2PasswordHasher`.

Controller classes implement `NavigatorAware`, and service interfaces are realized by concrete services in the service layer.

```java
public class Pbkdf2PasswordHasher implements PasswordHasher {
    @Override
    public String hash(String plainTextPassword) { ... }

    @Override
    public boolean matches(String plainTextPassword, String
        storedHash) { ... }
}
```

Listing 2: Inheritance – interface realized by concrete class

4.3 Polymorphism

Polymorphism is demonstrated when one concrete class perorms multiple tasks and services are dependent on interfaces rather than concrete repository implementations. `UserService` can be used via login and profile pages, while interface-based typing allows for repository substitution.

Applied evidence (4.3): a `UserService` instance is used through both `UserAuthService` and `UserProfileService` references; service constructors depend on repository interfaces (such as `UserRepository`) so `Jdbc*Repository` implementations can be swapped without changing service consumers.

```java
public class UserService implements UserAuthService,
   UserProfileService {

    private final UserRepository userRepository;
    private final PasswordHasher passwordHasher;

    public UserService(UserRepository userRepository, ...,
        PasswordHasher passwordHasher) {
         this.userRepository = userRepository;
         this.passwordHasher = passwordHasher;
    }

    @Override
    public ServiceResult<UserAccount> login(String email, String
        password) { ... }

    @Override
    public ServiceResult<UserAccount> register(String name, String
```

```
        email, String password) { ... }

    @Override
    public ServiceResult<UserAccount> getProfile(int userId) { ...
        }

    @Override
    public ServiceResult<UserAccount> updateProfile(int userId,
       String name, String email) { ... }
}
```

Listing 3: Polymorphism – UserService via dual interfaces

4.4 Abstraction

Abstraction is provided by `ServiceResult<T>` for success/failure handling, repository interfaces for data operations, and `ApplicationServices` for dependency wiring. Controllers consume high-level contracts without SQL or exception plumbing details.

Applied evidence (4.4): controllers receive `ServiceResult<T>` objects from services, repository interfaces expose operations without SQL details, and application wiring is hidden behind `ApplicationServices`.

```
// Replace with actual project snippet
public record ServiceResult<T>(boolean success, T data, String
   message) {
    public static <T> ServiceResult<T> ok(T data) { return new
       ServiceResult<>(true, data, null); }
    public static <T> ServiceResult<T> fail(String message) {
       return new ServiceResult<>(false, null, message); }
}
```

Listing 4: Abstraction evidence snippet

5 Design Patterns Used

5.1 Singleton Pattern

In `Database` and `SessionManager` (and `Jpa` if applicable), one single instance of DatabaseConnection and SessionManager is used to ensure that one shared instance is reserved for connection/session issues. Central state monitoring is now feasible, and it avoids uncontrolled duplicate

initializations. *Wikipedia (2025)*

5.2 Repository Pattern

Repository interfaces monitor persistence operations, while JDBC classes provide concrete SQL implementations. This distinguishes application logic from SQL, and it allows for future data-source substitutions without changing service contracts.

5.3 Service Locator Pattern

`ApplicationServices` is a service locator that centralizes wiring and inspection. The trade-off is reduced buildor boilerplate against increased static-state coupling, which may reduce test isolation. *Wikipedia (2025)*

5.4 Result Object (Railway-Oriented) Pattern

`ServiceResult<T>` has a detailed success/failure ratios and warnings, so that controllers can report on findings without having to rely on exception-driven control flow for anticipated company conditions. This promotes clearer flow and reduces the risk of unhandled exceptions.

5.5 Shell + Content UI Composition Pattern

`MainShellController's` switchable center content is used in a shell-content compositor style that is consistent with Composite or Strategy-like UI behavior. Navigation updates content while keeping traditional shell components intact.

6 Implementation – Core Features

This section explains selected features using focused code paths instead of full source dumps.

6.1 Feature 1: User Registration with Automatic Personal Group Creation

What it does: Registers a new user and creates a default personal group in the same business flow.

Classes involved: `UserService`, `GroupsService`, `JdbcUserRepository`, `JdbcGroupRepository`, and `ServiceResult<T>`.

Non-trivial logic: If the second persistence step isn't fruitful, please describe a uniqueness check, default group naming, and rollback/consistency behavior.

6.2 Feature 2: createExpenseWithSplits and Atomic Split Validation

What it does: Creates an expense and related splits only when the sum of split amounts equals the expense total.

Classes involved: `ExpensesService`, `JdbcExpenseRepository`, `JdbcExpenseSplitRepository`, `Validation`, and `ServiceResult<T>`.

```java
public ServiceResult<ExpenseModel> createExpenseWithSplits(...,
        BigDecimal amount, ..., List<ExpenseSplitModel> splits) {

    if (splits == null || splits.isEmpty()) {
        return ServiceResult.fail("At least one split is required.");
    }

    BigDecimal splitsTotal = splits.stream()
            .map(ExpenseSplitModel::shareAmount)
            .reduce(BigDecimal.ZERO, BigDecimal::add);

    if (splitsTotal.compareTo(amount) != 0) {
        return ServiceResult.fail(
            "Split amounts (" + splitsTotal + ") must equal expense amount (" + amount + ").");
    }

    ServiceResult<ExpenseModel> createResult = createExpense(...);
    if (!createResult.success()) return createResult;

    int expenseId = createResult.data().expenseId();
    for (ExpenseSplitModel split : splits) {
        int splitId = splitRepository.create(expenseId, split.userId(),
                split.shareAmount(), split.sharePercentage());
        if (splitId <= 0) {
            return ServiceResult.fail("Expense created but a split could not be saved.");
        }
    }

    return expenseRepository.findById(expenseId)
            .map(e -> ServiceResult.ok("Expense with splits created.", e))
```

```
            .orElseGet(() -> ServiceResult.fail("Expense created
                but could not be reloaded."));
}
```

Listing 5: Critical path: createExpenseWithSplits

Non-trivial logic: The sum-check uses `BigDecimal#compareTo` rather than `equals` to avoid scale mismatches (e.g. `10.0` vs `10.00`). Atomicity is application-level: if any split insert returns `id <= 0`, the method returns failure immediately, leaving the expense row orphaned – a deliberate trade-off in the absence of explicit transaction rollback at this layer.

6.3 Feature 3: Password Hashing and Timing-Safe Verification

What it does: Create secure hashes of credentials and verifies credentials using a timing-safe comparison strategy.

Classes involved: `Pbkdf2PasswordHasher`, `UserService`, `EnvConfig`.

Non-trivial logic: Explain salt use, iteration count configuration, and constant-time comparison rationale.

6.4 Feature 4: settleTransaction End-to-End Flow

What it does: Settles debt between members and updates balances/transaction history consistently.

Classes involved: `TransactionsService`, `ExpensesService`, repository interfaces and implementations.

Non-trivial logic: Explain validation of payer/payee, amount boundaries, and post-settlement ledger updates.

6.5 Feature 5: ViewNavigator Shell + Content Swap Mechanism

What it does: Keeps a persistent shell while replacing central views according to navigation events.

Classes involved: `ViewNavigator`, `MainShellController`, `NavigatorAware` controllers.

Non-trivial logic: Explain lifecycle ordering, controller injection, and session consistency constraints.

7 Testing

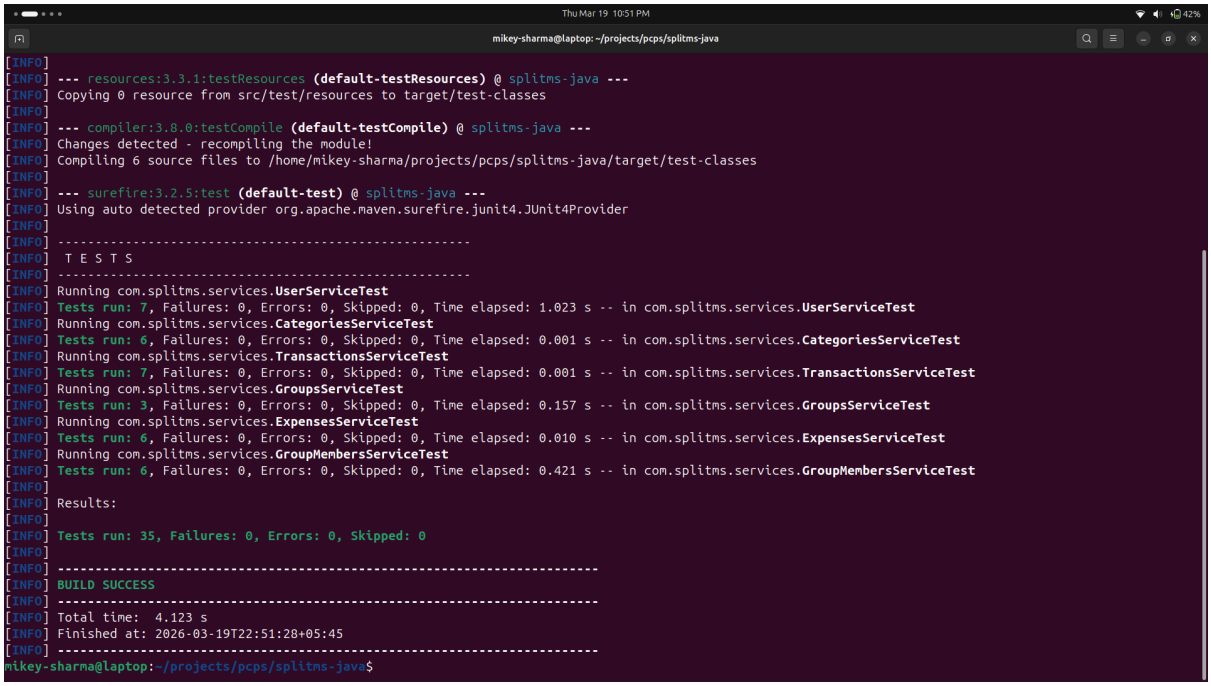
The current test suite is more oriented and runs against a live database. From start to finish, this has increased confidence in repository and service operations, but it is slower and less representative than simple unit tests.

7.1 Test Class Coverage Matrix

Table 3: Integration test coverage overview

Test Class	Service	Scenarios Covered
UserServiceTest	UserService	Login, wrong password, unknown email, duplicate registration, profile update
GroupsServiceTest	GroupsService	Create group, fetch by user, invalid metadata
GroupMembersServiceTest	GroupMembersService	Add/remove member, duplicate prevention, auth checks
ExpensesServiceTest	ExpensesService	Create expense, split validation, invalid shares, retrieval
TransactionsServiceTest	TransactionsService	Settle success, overpayment rejection
ViewNavigatorTest	ViewNavigator	View routing, shell replacement, navigation guards

7.2 Test Execution Evidence



```
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ splitms-java ---
[INFO] Copying 0 resource from src/test/resources to target/test-classes
[INFO]
[INFO] --- compiler:3.8.0:testCompile (default-testCompile) @ splitms-java ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 6 source files to /home/mikey-sharma/projects/pcps/splitms-java/target/test-classes
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ splitms-java ---
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.splitms.services.UserServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.023 s -- in com.splitms.services.UserServiceTest
[INFO] Running com.splitms.services.CategoriesServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.splitms.services.CategoriesServiceTest
[INFO] Running com.splitms.services.TransactionsServiceTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.splitms.services.TransactionsServiceTest
[INFO] Running com.splitms.services.GroupsServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.157 s -- in com.splitms.services.GroupsServiceTest
[INFO] Running com.splitms.services.ExpensesServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in com.splitms.services.ExpensesServiceTest
[INFO] Running com.splitms.services.GroupMembersServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.421 s -- in com.splitms.services.GroupMembersServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 35, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.123 s
[INFO] Finished at: 2026-03-19T22:51:28+05:45
[INFO] ------------------------------------------------------------------------
mikey-sharma@laptop:~/projects/pcps/splitms-java$
```

Figure 4: Full test suite execution output (green/passing)

7.3 Testing Limitation and Improvement Note

These are integration tests that necessitate a live database as well as a traditional schema. To ensure service logic can be tested without the use of databases, a true unit-test system would need mocked repository interfaces and deterministic fake data providers.

8 Challenges and Solutions

Using the right problem attempt solution learning framework, at least two main implementation issues have been identified below.

8.1 Challenge 1: Atomic Split Creation Without Full Transaction Framework

Problem: `createExpenseWithSplits` required all rows to persist together or none at all.

What was tried: Naive multi-step insertions resulted in risk of partial data on intermediate failures.

Solution: Introduced service-level orchestration with solid pre-validation, explicit failure

detection and handling, and transactional boundary controls in persistence operations.

What was learned: Before writing services, data consistency must be followed, and service atomicity issues must be addressed.

8.2 Challenge 2: JavaFX Shell + Content Lifecycle Coordination

Problem: When storing session state caused lifecycle order issues, coordinating scene swaps were helpful.

What was tried: The direct scene replacement resulted in inconsistent session-dependent UI behavior.

Solution: Adopted shell-content architecture with `ViewNavigator` and controlled content injection into `MainShellController`.

What was learned: Coordinated navigation policies have reduced coupling and increased predictable UI state transitions.

References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley.

Wikipedia (2025) 'Singleton pattern'. Available at: https://en.wikipedia.org/wiki/Singleton_pattern (Accessed: 19 March 2026).

Wikipedia (2025) 'Service locator pattern'. Available at: https://en.wikipedia.org/wiki/Service_locator_pattern (Accessed: 19 March 2026).

Dhawan, K. (2026) 'New Component: Shell, a Layout Engine for Modern Apps'. https://dev.to/kushagradhawan/about-shell-156o (Accessed: 19 March 2026).

A Appendix: Submission Repository

GitHub submission link: https://github.com/MikeySharma/javafx-splitms/tree/submission/week-9